

Multi-User Chat Application Using Linux System Calls

Process Management, Inter-Process Communication and TCP Socket Programming

Team 15 | Section S-04 · Ch. Yashwanth · V. Bharath · B. Praneeth · Sep 29, 2026

Course and Team Details

This report documents the OSSP project of Team 15, Section S-04, built as a TCP chat system in C on Ubuntu.

Item	Details
Institution	Koneru Lakshmaiah Education Foundation (KLEF)
Course	Operating Systems and Systems Programming (OSSP)
Academic year	2026–27, Term-I
Team	Team 15, Section S-04
Faculty	Dr. Soumya

Team member	Roll number
Ch. Yashwanth	2520030259
V. Bharath	2520030169
B. Praneeth	2520030186

Abstract

The project is a multi-user chat system written in C on Linux that shows how system calls, processes, pipes and sockets work together.

A central server listens on a TCP port and accepts up to 32 clients. For every client, the server calls `fork()` to create a child process. Each child handles its client's socket and sends events to the parent server through a `pipe()`. The parent owns the client table and performs all routing: login, online-user listing, private messages and broadcasts.

Signals handle child cleanup (`SIGCHLD`) and clean shutdown (`SIGINT`). The result is a small but complete example of process management, inter-process communication and network programming on Linux.

Problem Statement and Objectives

Simple chat demonstrations often hide the operating-system mechanisms behind process creation, communication and resource management. This project asks: how can Linux system calls be combined to build a reliable multi-user communication system?

The system must meet these needs:

- Multiple users communicate at the same time.
- The server handles many client connections.
- Each client is handled by its own process.
- Processes exchange data through an IPC mechanism.
- TCP sockets give reliable communication.
- Connections and resources are created, managed and closed safely.

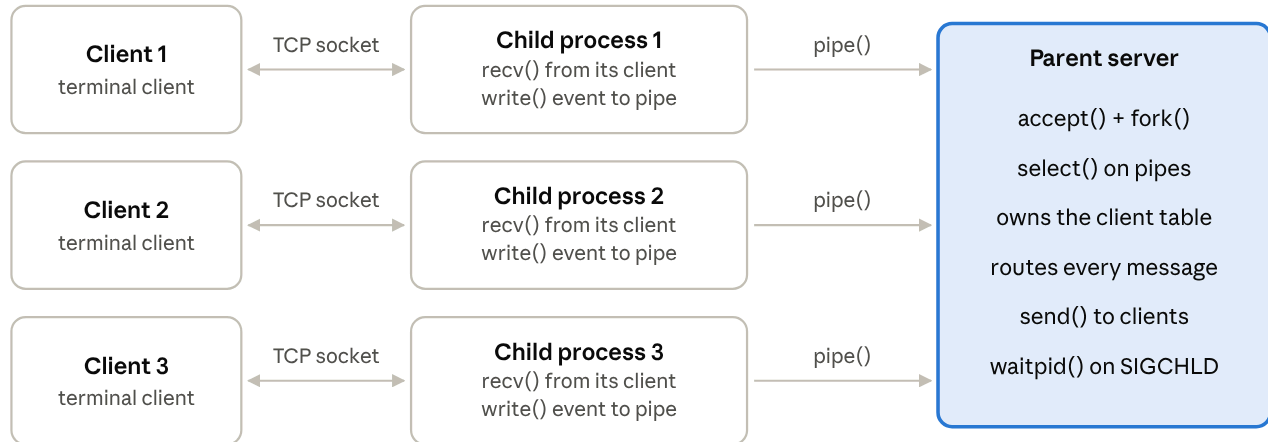
Objectives

1. Demonstrate process creation with `fork()`.
2. Demonstrate IPC with `pipe()`.
3. Demonstrate TCP socket programming.
4. Demonstrate signal handling and child cleanup.
5. Support multiple simultaneous clients.

System Architecture

The server is one parent process plus one child process per connected client.

Each client gets its own child; only the parent routes messages



system architecture · parent server, 3 child processes, 3 clients

Each child reads its client's socket and writes events into a pipe. Only the parent updates the client table, routes messages and replies on the client sockets.

Linux System Calls Used

The project uses 18 system calls, listed here by where they appear in the code.

Call	Purpose	Used in
<code>socket()</code>	Create a TCP endpoint	Server, client
<code>setsockopt()</code>	Allow quick port reuse (<code>SO_REUSEADDR</code>)	Server
<code>bind()</code>	Attach the socket to a port	Server
<code>listen()</code>	Start waiting for connections	Server
<code>accept()</code>	Accept a client, return a new socket	Server
<code>connect()</code>	Connect to the server	Client
<code>fork()</code>	Create one child process per client	Server
<code>pipe()</code>	Create the child-to-parent IPC channel	Server
<code>read()</code> / <code>write()</code>	Move events through the pipe	Parent / child
<code>send()</code> / <code>recv()</code>	Move chat text through sockets	Server, client

Call	Purpose	Used in
<code>select()</code>	Wait on the listening socket, pipes and keyboard	Server, client
<code>close()</code>	Release sockets and pipe ends	Server, client
<code>waitpid()</code>	Reap finished children	Server
<code>sigaction()</code>	Install <code>SIGINT</code> and <code>SIGCHLD</code> handlers	Server, client
<code>kill()</code>	Stop child processes	Server
<code>getpid()</code>	Print the server PID	Server

Sockets and pipes are file descriptors, which is why `read()`, `write()` and `close()` work on both.

Implementation Details

The parent server owns all shared state, and every child only relays what its client types.

Process management. The parent runs a `select()` loop over the listening socket and each child's pipe. When a client connects, the parent finds a free slot, calls `pipe()`, then `fork()`. The child closes the listening socket and serves its client. The parent closes the pipe's write end and records the client in its table.

IPC design. After `fork()`, parent and child have separate memory, so a child cannot edit the parent's `clients[]` array. Instead the child writes a fixed-size `server_event_t` (login, message, private message, user list, quit) into the pipe. The parent reads it, updates its table and sends replies on the client sockets it owns.

Signals and cleanup.

- `SIGCHLD`: the handler calls `waitpid(-1, WNOHANG)` in a loop so finished children never stay as zombies.
- `SIGINT`: sets a flag; the server then notifies clients, terminates children, closes descriptors and waits for all children.
- `SIGPIPE`: ignored, and `send()` uses `MSG_NOSIGNAL`, so a dropped client cannot crash the server.

Error handling. The return values of the main system calls are checked. A failed `accept()`, `pipe()` or `fork()` closes the descriptors it opened and the server keeps running. Usernames are validated (letters, digits, `_`, `-`) and must be unique.

Chat commands

Command	Action
<code>/help</code>	Show the command list
<code>/users</code>	List online users
<code>/msg <user> <message></code>	Private message
<code>/broadcast <message></code>	Message to all users
<code>/quit</code>	Leave the chat

Any other text is broadcast to everyone.

How to Build and Run

The project builds with `make` on Ubuntu or WSL2 and needs only GCC and Make.

1. Install tools

```
sudo apt update
sudo apt install build-essential
```

2. Build (compiled with `-Wall -Wextra -Wpedantic -std=c11`; no warnings)

```
cd multi_user_chat
make
```

3. Start the server (Terminal 1)

```
./bin/server 8080
```

4. Start clients (three more terminals; use a different username in each)

```
./bin/client 127.0.0.1 8080
```

Ports must be between 1024 and 65535. Press Ctrl+C in the server terminal to shut down.

Testing and Validation

The server and clients were compiled with strict warnings and tested with three simultaneous clients on Linux; all features passed, and one shutdown bug was found and fixed.

Tests run

Test	Expected	Result
Three clients log in	All accepted, one child process each	Pass
Normal message	Every other user receives it	Pass
<code>/msg bob secret</code>	Only bob (and sender) see it	Pass
<code>/users</code>	Numbered list of online users	Pass
Client drops suddenly	Others told the user left; server keeps running	Pass
<code>/quit</code>	User removed, no zombie process	Pass
Duplicate username	Login rejected with an error	Pass
Ctrl+C after a rejected login	Server exits cleanly	Failed, then fixed

Bug found and fix. When a login was rejected, the parent removed the client but its child process kept running. On Ctrl+C the server then waited forever in `waitpid()`. The parent also broadcast a blank "left the chat" message for a user who never logged in. The fix in `remove_client()` announces the departure only when a username is set, and calls `kill(pid, SIGTERM)` on the child before closing the socket:

```
if (clients[index].username[0] != '\0') {
    char notice[MAX_MESSAGE + 80];
    snprintf(notice, sizeof(notice), "[SERVER] %s left the chat.\n",
             clients[index].username);
    printf("%s", notice);
    broadcast_text(notice, index);
}
if (clients[index].pid > 0)
    kill(clients[index].pid, SIGTERM);
close(clients[index].socket_fd);
```

After the fix, the duplicate-username case left one child alive and the server exited with code 0 on Ctrl+C.

Validation against objectives

Objective	Evidence
Process creation with <code>fork()</code>	One child process per connected client
IPC with <code>pipe()</code>	Parent receives events and routes messages
TCP sockets	Clients connect over TCP port 8080
Signal handling and cleanup	No zombies; Ctrl+C shuts down cleanly
Multiple simultaneous clients	Three clients chatting at once

Debugging commands used: `ps -ef --forest`, `ss -tlnp | grep 8080`, `strace -f ./bin/server 8080`.

Team Contributions

The work was divided among Ch. Yashwanth, V. Bharath and B. Praneeth; write each member's name beside their area in the table below.

Area of work	Team member
Server development: sockets, <code>accept()</code> , <code>fork()</code> , event routing	(add name)
Client development: connection, <code>select()</code> loop, user input	(add name)
Linux system-call integration	(add name)
Signal handling and process cleanup	(add name)
Error handling and debugging	(add name)
Documentation	(add name)
Testing and project validation	(add name)

Conclusion and Future Scope

The project shows that `fork()`, `pipe()`, sockets and signals can be combined into a working multi-user chat system in about 600 lines of C.

Each child process serves one client, pipes carry events to the parent, TCP sockets carry the chat text, and signals with `close()` and `waitpid()` manage the process lifecycle. It is meant for local and college demonstrations, not production use.

Future scope

- Authentication and TLS encryption
- Persistent chat history
- Better message framing in the protocol
- Scalable event-driven I/O with `epoll()`
- A graphical client